

LAB 2 – CAYENNE CODING AND API KEYS

Internet of Things and Smart Cities



Setup the library

- Install the CayenneLPP library
- Download the «cayenne.ino» file from Moodle
- Open «cayenne.ino» in the Arduino IDE

```
1 // Before running edit properly line 42 and 4
2
3 #include <MKRWAN.h>
4 ////////////////////////////////////
5 LoRaModem modem;
6
7 //edit with your own appEui and appKey
8 String appEui = "";
9 String appKey = "";
10
11 String messages [4] = {"Welcome to IoT Lab",
12 int i = 0;
13
14 void setup() {
15     int connected;
16     Serial.begin(115200);
17     while (!Serial);
18
19     if (!modem.begin(EU868)) {
```



```
1 // Before running edit properly line 42 and 4
2
3 #include <MKRWAN.h>
4+ #include <CayenneLPP.h>
5
6 LoRaModem modem;
7
8 //edit with your own appEui and appKey
9 String appEui = "";
10 String appKey = "";
11
12+ CayenneLPP lpp(51);
13 ////////////////////////////////////
14
15 void setup() {
16     int connected;
17     Serial.begin(115200);
18     while (!Serial);
19
20     if (!modem.begin(EU868)) {
```

```

    Serial.print("\n");
}

// ...

void loop() {
    Serial.println("Waiting 1 minute to send next
    wait(60);
    while (modem.available()) {
        Serial.write(modem.read());
    }
    modem.poll();

    int err;
    modem.setPort(3);
    modem.beginPacket();

    modem.print(messages[i%4]);
    err = modem.endPacket(true);
    if (err > 0) {
        i = i+1;
        Serial.println("Message sent correctly!");
    } else {
        Serial.println("Error sending message :(");
    }
}

```



```

43     Serial.print("\n");
44 }
45+
46+ void printVariables(){
47+     lpp.reset();
48+     // add your variables here
49+     // ...
50+     // ...
51+ }
52
53 void loop() {
54     Serial.println("Waiting 1 minute to send next message");
55     wait(60);
56     while (modem.available()) {
57         Serial.write(modem.read());
58     }
59     modem.poll();
60
61     int err;
62     modem.setPort(3);
63     modem.beginPacket();
64+     printVariables();
65+     modem.write(lpp.getBuffer(), lpp.getSize());
66     err = modem.endPacket(true);
67     if (err > 0) {
68         Serial.println("Message sent correctly!");
69     } else {
70         Serial.println("Error sending message :(");
71     }
72 }

```



Task 1

- Create a new application on TTN (you can refer to the slides of lab 1 if needed)
- Edit **appEui** and **appKey** to establish the connection (you can refer to the slides of lab 1 if needed)



Task 2

- Edit the `printVariables()` function to buffer:
 - A relative humidity variable with values that are uniform distributed in the range [40, 50] with precision of 0.05 on channel 3
 - A temperature variable with values that are uniform distributed in the range [15, 35] with precision of 0.01 on channel 4
 - A pressure variable with values that are uniform distributed in the range [1013, 1033] with precision 0.01 on channel 7
 - A pH variable with values that are uniform distributed in the range [6.99, 7.01] with precision of 0.001 on channel 8

You should use the `random(min, max)` function of Arduino that gives a random integer between **min** and **max**



Setup TTN 1

- Change the payload formatter of the device (use CayenneLPP)
- Once this operation is done you should see your messages that are correctly decoded in the live messages
- Enable the Message Storage
- Create a new API key

device-00

ID: device-00

+ Add label



Device overview



Live data



Messaging



Location

Uplink

Downlink

Setup

Formatter type *

CayenneLPP



 [What is CayenneLPP?](#)

Test

Byte payload

Decoded test payload

--	--

TIME	TYPE	DATA PREVIEW	Verbose stream	Export as JSON		
↓ 13:24:47	Schedule data downlink for transmi...	DevAddr: 26 0B CE 1A Rx1 Delay: 5				
↑ 13:24:47	Forward uplink data message	DevAddr: 26 0B CE 1A Payload: { analog_in_8: 24.29, barometric_pressure_7: 26.2, relative_humidity_3: 66.5, temperature_4: 24.3 }			03 68 85 04 67 00 F3 0	
↑ 13:24:47	Successfully processed data message	DevAddr: 26 0B CE 1A				
↑ 13:23:41	Forward join-accept message	DevAddr: 26 0B CE 1A JoinEUI: 00 00 00 00 00 00 00 01 DevEUI: A8 61 0A 34 32 20 73 09				
↑ 13:23:39	Successfully processed join-request	DevAddr: 26 0B 77 48 JoinEUI: 00 00 00 00 00 00 00 01 DevEUI: A8 61 0A 34 32 20 73 09				
🔗 13:23:39	Accept join-request	DevAddr: 26 0B CE 1A JoinEUI: 00 00 00 00 00 00 00 01 DevEUI: A8 61 0A 34 32 20 73 09				



Setup TTN 2

- Enable the Message Storage
- Open the Message Storage tab on the menu on the left
- Check the checkbox to activate the Storage
- Create a new API key

Home

Applications

Gateways

Q Search

Ctrl

K



Second Lab

Application overview

End devices

Live data

Webhooks

Message storage

Payload formatters

Collaborators

MAC settings profiles

API keys

Other integrations

General settings

Top end devices

device-00

Second Lab

+ Add label

ID: 2nd-lab-iot-fb



This page is experimental and subject to change

Message storage

Message storage allows storing received upstream messages in a persistent database, and retrieving them at

Status • (currently activated)Activate message storage ☒

Since a lot of data can be retrieved causing poor performance, we only fetch the first 10 items from th

All devices



Last 12 hours



All devices stored messages

END DEVICE ID

RECEIVED

FPORT

FCNT

SI

No items

Download



Setup TTN 3

- Create a new API key
 - Open the API keys tab on the menu on the left
 - Choose a name for the API key
 - Generate the key
 - You need to copy and paste the key before closing the creation pop-up, because your key will not be shown elsewhere

Home

Applications

Gateways

Search

Ctrl

K



Second Lab

Application overview

End devices

Live data

Webhooks

Message storage

Payload formatters



Collaborators

MAC settings profiles

API keys

Other integrations



General settings

Top end devices



device-00

Second Lab

+ Add label

ID: 2nd-lab-iot-fb

Add API key

Name

My new API key

Expiry date

mm / dd / yyyy



Rights*

☒ Grant all current and future rights☐ Grant individual rights☐ Select all☐ delete application☐ view devices in application☐ view device keys in application☐ create devices in application☐ edit device keys in application☐ view application information☐ link as Application to a Network Server for traffic exchange, i.e. read uplink and write downlink

This implicitly includes the rights to view application information, read application traffic and write downlinks

☐ purge application

Please copy newly created API key

You won't be able to view the key afterward

Granted rights

- ✓ All application rights

Your API key has been created successfully. Note: After closing this window, the value of the key secret will not be accessible anymore. Make sure to copy and store it in a safe place now.

API key

..... 

✓ I have copied the key



Populate the storage

- Once this procedure is done, you should make your Arduino send some messages, to have some messages stored in the database for the next phase

Second Lab

+ Add label

ID: 2nd-lab-iot-fb

📶 Last activity just now • 🖨 0 End devices

☆ ☰

TIME	ENTITY ID	TYPE	DATA PREVIEW	Verbose stream	Export as JSON	00	Clear
↑ 16:18:26	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.54, barometric_pressure_7: 1013.9, relative_humidity_3: 48, temperature_4: 25.8 }	☐	Export as JSON	00	Clear
↑ 16:18:20	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.31, barometric_pressure_7: 1029.3, relative_humidity_3: 41.5, temperature_4: 24.7 }	☐	Export as JSON	00	Clear
↑ 16:18:15	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 9.3, barometric_pressure_7: 1020.5, relative_humidity_3: 41, temperature_4: 31.3 }	☐	Export as JSON	00	Clear
↑ 16:18:10	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 7.57, barometric_pressure_7: 1017.4, relative_humidity_3: 47.5, temperature_4: 24.8 }	☐	Export as JSON	00	Clear
↑ 16:18:05	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 7.15, barometric_pressure_7: 1028.6, relative_humidity_3: 47, temperature_4: 21.5 }	☐	Export as JSON	00	Clear
↑ 16:17:59	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.41, barometric_pressure_7: 1031.8, relative_humidity_3: 49, temperature_4: 23 }	☐	Export as JSON	00	Clear
↑ 16:17:54	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 7.75, barometric_pressure_7: 1021.6, relative_humidity_3: 49.5, temperature_4: 26.6 }	☐	Export as JSON	00	Clear
↑ 16:17:49	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 10.01, barometric_pressure_7: 1018.2, relative_humidity_3: 48.5, temperature_4: 17.9 }	☐	Export as JSON	00	Clear
↑ 16:17:43	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.55, barometric_pressure_7: 1019.9, relative_humidity_3: 45.5, temperature_4: 27.5 }	☐	Export as JSON	00	Clear
↑ 16:17:37	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.15, barometric_pressure_7: 1027.4, relative_humidity_3: 44.5, temperature_4: 28.6 }	☐	Export as JSON	00	Clear
↑ 16:17:32	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 8.57, barometric_pressure_7: 1032.5, relative_humidity_3: 42.5, temperature_4: 30.2 }	☐	Export as JSON	00	Clear
↑ 16:17:27	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 9.39, barometric_pressure_7: 1021.2, relative_humidity_3: 44, temperature_4: 28.4 }	☐	Export as JSON	00	Clear
↑ 16:17:20	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 9.08, barometric_pressure_7: 1030, relative_humidity_3: 42, temperature_4: 18.4 }	☐	Export as JSON	00	Clear
↑ 16:17:14	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 9.61, barometric_pressure_7: 1018.3, relative_humidity_3: 45.5, temperature_4: 33 }	☐	Export as JSON	00	Clear
↑ 16:17:08	device-00	Forward uplink data message	DevAddr: 26 0B 00 3A Payload: { analog_in_8: 9.74, barometric_pressure_7: 1020.2, relative_humidity_3: 42, temperature_4: 20.2 }	☐	Export as JSON	00	Clear

Setup python

- Download `data-retrieval.py` (or `data-retrieval.ipynb` if you prefer)
- Edit the following variables:

Variable	Value
<code>api_key</code>	The string you copied in the pop-up
<code>application_id</code>	The string that identifies your application, is the one you choose while creating the app
<code>device_id</code>	If None all messages will be downloaded, otherwise you can specify the string that identifies your device, is the one you choose while registering the device
<code>limit</code>	The maximum number of messages to download (I suggest to set it to an high number)
<code>after</code>	Timestamp of the arrival of the first downloaded message

```

27     for line in response.split("\n"):
28         if line.strip():
29             objs.append(json.loads(line))
30     return objs

```

[]

Python

```

1  api_key = ""
2  application_id = ""
3  device_id = ""
4  limit = 0
5  after = "2020-10-10T10:10:10Z"
6  field_mask = "up.uplink_message.decoded_payload"
7  uplink_data = get_uplinks(api_key, application_id, device_id, limit, after, field_mask)
8  data = []
9  for d in uplink_data:
10     try:
11         dd = d["result"]["uplink_message"]["decoded_payload"]
12         dd["time"] = d["result"]["uplink_message"]["received_at"]
13         data.append(dd)
14     except:
15         pass
16  df = pd.DataFrame(data)
17  df["time"] = pd.to_datetime(df["time"])
18  df.sort_values("time", inplace=True)
19
20  display(df)

```

[]

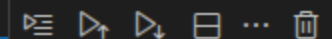
Python



```

1  df.rename(columns={"temperature_4": "Temperature (°C)",
2                    "analog_in_8": "pH",
3                    "barometric_pressure_7": "Pressure (hPa)",
4                    "relative_humidity_3": "Humidity (%)",

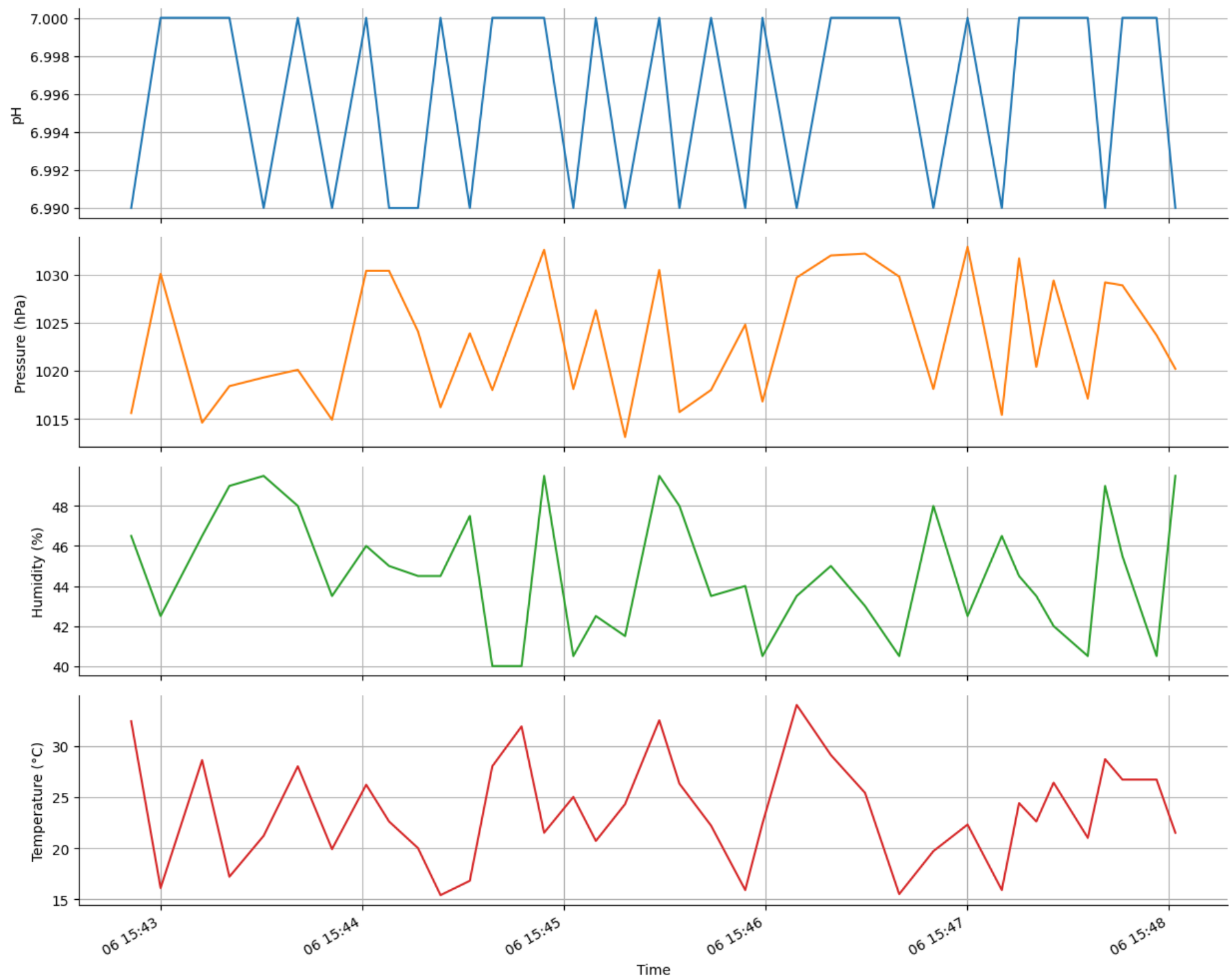
```





Execute the python code

- You should see a plot of the downloaded data





Task 3

- If you look closely to the pH plot you will notice that the pH assumes only values that are 6.99, 7.00, or 7.01, even thou in the Arduino we specified a precision of 0.001.
- Find why this happens
- Change the Arduino code and the Python code to have the desired precision (of 0.001) also in the plot